

# TRABAJO PRÁCTICO

Caso de estudio — Gestión de Residuos Urbanos


**Alumnos:**

Pepino, Facundo

Truchet, Mauricio

**Docentes:**

Ing. Bono, Juan Pablo


**Asignatura:**

Arquitecturas Móviles

**Ciclo Lectivo:**

2026

# ÍNDICE

1. CONSIGNA ..... 3
2. INTRODUCCIÓN ..... 5
3. REQUERIMIENTOS DEL SISTEMA ..... 5
3.1. Requerimientos funcionales ..... 5
3.2. Requerimientos no funcionales ..... 6
3.3. Alternativas de desarrollo móvil evaluadas ..... 7
4. DISEÑO DE ARQUITECTURA ..... 8
4.1. Estructura general de la solución ..... 8
4.2. Flujo de trabajo y secuencia operativa ..... 9
4.3. Modelo de persistencia y datos en Cloud Firestore ..... 10
4.4. Algoritmo de optimización de rutas de recolección ..... 11
4.5. Sensores en los contenedores y protocolo de comunicación ..... 12
5. PROTOTIPO FUNCIONAL ..... 13
5.1. Módulo de autenticación, perfiles y control de acceso ..... 13
5.2. Módulo ciudadano ..... 14
5.3. Panel de control para autoridades municipales ..... 14
5.4. Módulo operativo para personal de cuadrilla ..... 15
5.5. Capturas de la interfaz en ejecución ..... 15
5.6. Plataforma de cartografía y geolocalización ..... 16
6. ESCALADO Y MONITOREO ..... 17
6.1. Contenerización de servicios con Docker ..... 17
6.2. Orquestación y balanceo con Kubernetes ..... 17
6.3. Autoescalado horizontal de réplicas ..... 18
6.4. Observabilidad y monitoreo con Prometheus y Grafana ..... 19
7. ESTRATEGIA DE PRUEBAS Y CALIDAD ..... 20
8. PROBLEMAS Y SOLUCIONES ..... 21
9. CUMPLIMIENTO DE LA CONSIGNA ..... 22
10. INNOVACIÓN Y PARTICIPACIÓN ..... 24
11. CONCLUSIONES ..... 24
12. BIBLIOGRAFÍA ..... 24

# 1. CONSIGNA

Caso de estudio Web Mobile App Gestión Residuos. Trabajo práctico de desarrollo de un sistema de gestión de residuos vecinales para la ciudad de San Francisco.

Contexto.

La ciudad de San Francisco necesita implementar un sistema de gestión de residuos vecinales que permita optimizar la recolección de desechos, mejorar el control de residuos especiales (reciclables, voluminosos, peligrosos) y fomentar la participación ciudadana en el cuidado ambiental.

La solución debe ser eficiente y escalable, involucrando tanto a los ciudadanos como a las autoridades locales y al personal de recolección.

Objetivo del trabajo práctico.

El objetivo de este trabajo es diseñar y prototipar un sistema de aplicaciones móviles y servicios en la nube para gestionar la recolección de residuos en la ciudad de San Francisco.

El sistema deberá contar con aplicaciones específicas para los ciudadanos, autoridades y personal de recolección, utilizando herramientas de escalado y optimización en tiempo real para satisfacer la demanda de una ciudad en crecimiento.

Requerimientos.

- Aplicación ciudadana. Desarrollar una aplicación móvil para los ciudadanos con las siguientes funcionalidades. Registro de residuos generados (categoría y cantidad). Solicitud de recolección para residuos especiales (reciclables, voluminosos, peligrosos). Acceso a la ubicación de puntos de reciclaje y horarios de recolección. Recepción de notificaciones sobre campañas de reciclaje o eventos de limpieza.
- Aplicación de autoridades. Crear una aplicación o panel de control para las autoridades que permita la visualización de los puntos de residuos a recolectar y su estado en tiempo real. Planificación y asignación de rutas de recolección para el personal. Monitoreo del estado de los puntos de recolección y gestión de incidentes.
- Aplicación para el personal de recolección. Visualización de las rutas de recolección asignadas y confirmación de tareas completadas. Escaneo de códigos QR o ingreso de identificadores para registrar la recolección en puntos específicos. Reportes sobre el estado de los contenedores o incidentes en el servicio.
- Plataforma de mapeo y geolocalización. Integración con una API de geolocalización (Google Maps, Mapbox, etc.) para localizar los puntos de recolección y generar rutas optimizadas en tiempo real.

- Notificaciones y escalabilidad. Implementar un sistema de notificaciones push para ciudadanos y personal de recolección. Diseñar la arquitectura para ser escalable usando contenedores (Docker) y orquestación (Kubernetes). Utilizar herramientas de monitoreo y autoescalado para asegurar el rendimiento y la disponibilidad del sistema bajo distintas cargas de uso.

Entregables.

- Diagrama de arquitectura. Un diagrama detallado que describa la arquitectura de la solución, mostrando la interacción entre los diferentes componentes (frontend, backend, bases de datos, APIs de geolocalización, y servicios de escalado).
- Prototipo funcional. Prototipo de la aplicación ciudadana, que permita probar el flujo de registro y solicitud de recolección. Prototipo de la aplicación para autoridades y personal de recolección, con funcionalidades básicas de monitoreo y asignación de tareas.
- Documentación técnica. Explicación detallada de las decisiones arquitectónicas. Justificación de las herramientas y servicios de escalado elegidos (Docker, Kubernetes, etc.). Descripción del flujo de trabajo y las rutas optimizadas de recolección de residuos.
- Esquema de escalado y monitoreo. Describir las estrategias de escalabilidad y monitoreo implementadas (uso de balanceadores de carga, autoescalado, monitoreo con Grafana/Prometheus o similar).

Criterios de evaluación.

- Diseño de arquitectura. Claridad, escalabilidad, y justificación de las herramientas seleccionadas.
- Funcionalidad del prototipo. Cumplimiento de las funcionalidades especificadas para cada rol.
- Documentación técnica. Explicación y justificación clara de cada componente y flujo de trabajo.
- Innovación y creatividad. Propuestas para mejorar la eficiencia del sistema o ideas adicionales para aumentar la participación ciudadana.

Etapas de entrega.

- Entrega del diagrama de arquitectura y justificación.
- Prototipo funcional y documentación técnica.
- Esquema de escalado y monitoreo.

# 2. INTRODUCCIÓN

La gestión de residuos de San Francisco, en la provincia de Córdoba, enfrenta barrios cada vez más dispersos y un servicio de recolección que todavía se organiza de forma manual. La solución propone una plataforma digital única, en la que vecinos, municipio y cuadrillas trabajan sobre la misma información y en tiempo real.

La plataforma atiende a tres actores.

- Los vecinos registran los residuos que generan, piden el retiro de materiales especiales y consultan los ecopuntos más cercanos, sus horarios y las campañas vigentes. Un ecopunto es un punto de reciclaje con un contenedor equipado con un sensor de llenado.
- La autoridad municipal supervisa el estado de los contenedores, reparte el trabajo entre las cuadrillas y resuelve los reclamos.
- Las cuadrillas recorren rutas optimizadas, confirman cada parada con un código QR y reportan problemas en la calle al instante.

La arquitectura suma recursos de forma automática ante picos de demanda, como los que siguen a una tormenta o a una jornada de limpieza.

# 3. REQUERIMIENTOS DEL SISTEMA

## 3.1. Requerimientos funcionales

El siguiente cuadro detalla las capacidades que debe ofrecer el sistema a cada perfil de usuario.

| Identificador | Perfil | Descripción funcional | Prioridad |
|---|---|---|---|
| RF-01 | Vecino | Registrar los residuos que genera, con su categoría y su cantidad, sin necesidad de pedir un retiro | Alta |
| RF-02 | Vecino | Solicitar el retiro de residuos reciclables, voluminosos y peligrosos | Crítica |
| RF-03 | Vecino | Ver en un mapa los ecopuntos con su dirección, horario de atención y nivel de llenado | Alta |
| RF-04 | Vecino | Recibir avisos en el celular sobre campañas ambientales y avances de sus pedidos | Media |
| RF-05 | Autoridad | Seguir en tiempo real las solicitudes pendientes, en curso y recolectadas | Crítica |

| Identificador | Perfil | Descripción funcional | Prioridad |
|---|---|---|---|
| RF-06 | Autoridad | Calcular una ruta optimizada para las solicitudes elegidas y asignarla a una cuadrilla | Crítica |
| RF-07 | Autoridad | Supervisar el llenado de los contenedores medido por sensores y recibir alertas de desborde | Crítica |
| RF-08 | Autoridad | Atender los incidentes reportados por vecinos y cuadrillas hasta su resolución | Alta |
| RF-09 | Autoridad | Enviar avisos de concientización a vecinos y cuadrillas | Media |
| RF-10 | Cuadrilla | Consultar la hoja de ruta con las paradas del turno, ordenadas y trazadas por las calles | Crítica |
| RF-11 | Cuadrilla | Validar cada parada con el código QR del contenedor o con el código de la solicitud | Alta |
| RF-12 | Cuadrilla | Confirmar el peso recolectado y registrar observaciones de la parada | Alta |
| RF-13 | Cuadrilla | Reportar problemas en la vía pública, como contenedores dañados o calles cortadas | Alta |
| RF-14 | Todos | Ingresar con la cuenta de Google y acceder solo a lo que permite su perfil | Crítica |

Requerimientos funcionales por perfil y prioridad.

## 3.2. Requerimientos no funcionales

Los atributos de calidad son los objetivos que guían la estructura del software y las decisiones de infraestructura.

- Disponibilidad. El servicio debe estar activo el 99,9% del año, es decir, menos de nueve horas de interrupción acumulada.
- Rapidez. El 95% de las consultas y los registros debe responder en menos de 250 milisegundos.
- Escalabilidad. La infraestructura suma réplicas automáticamente, de 2 a 10, cuando el uso del procesador supera el 70%.
- Tiempo real. Los cambios de estado de una solicitud aparecen en los paneles conectados en menos de dos segundos.
- Cobertura irregular. La aplicación se abre sin conexión y sincroniza los datos al recuperar la señal.
- Seguridad. Toda la comunicación viaja cifrada y el servidor exige los permisos de cada perfil.

- Uso en celulares. Las pantallas son táctiles, desde 360 píxeles de ancho, y cumplen criterios de accesibilidad.
- Peso reducido. La primera carga ocupa menos de 2 megabytes para que sea ágil en redes celulares.
- Sensores. Las lecturas de llenado se reciben con entrega confirmada en menos de 500 milisegundos.

## 3.3. Alternativas de desarrollo móvil evaluadas

Se compararon tres formas de construir la aplicación según su costo, su tiempo de puesta en marcha y la experiencia de uso. La aplicación web instalable cubre todos los requerimientos, porque el navegador accede a la ubicación y a la cámara del teléfono.

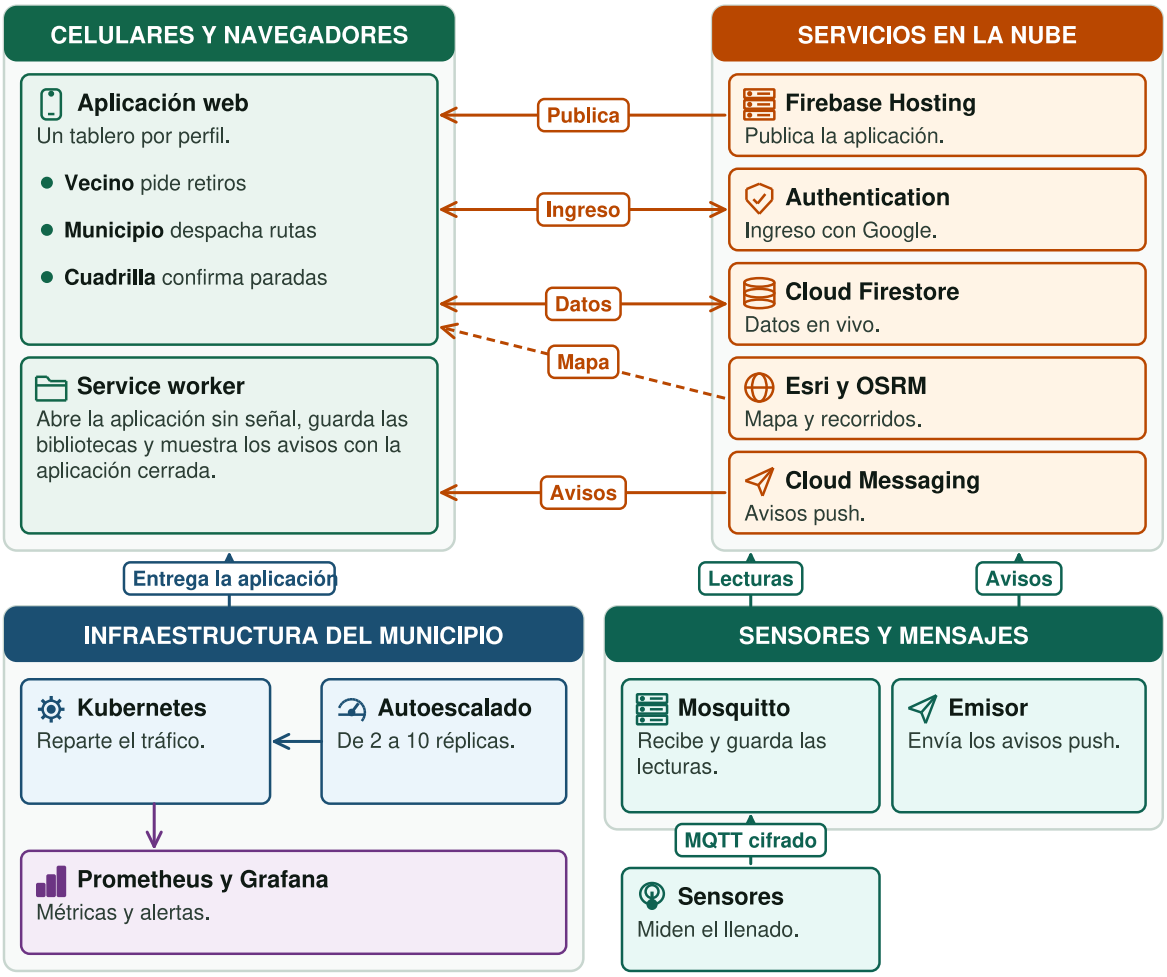
| Alternativa | Ventajas | Desventajas | Decisión |
|---|---|---|---|
| Aplicación web instalable | Se publica al instante sin pasar por las tiendas, funciona sin conexión y consume pocos datos | Acceso más limitado a algunos componentes del teléfono | Elegida por llegar a cualquier vecino con bajo costo de mantenimiento |
| Aplicación híbrida, con código web dentro de una aplicación de tienda | Se instala desde la tienda y accede a funciones del teléfono mediante complementos | Compilación más compleja y demoras de aprobación en las tiendas | Descartada porque exige descargar la aplicación para trámites ocasionales |
| Aplicación nativa, programada por separado para Android y para iOS | Mejor rendimiento en animaciones y acceso directo al sistema del teléfono | Hay que mantener dos programas distintos, con un costo de desarrollo alto | Descartada por presupuesto y plazos |

Comparación de las alternativas para construir la aplicación.

# 4. DISEÑO DE ARQUITECTURA

## 4.1. Estructura general de la solución

La arquitectura se organiza en cuatro bloques. Cada uno cumple una función propia y puede cambiar sin afectar a los demás.




Componentes del sistema y relación entre ellos.

- Los celulares y navegadores ejecutan la aplicación web. Un service worker, un programa que el navegador mantiene en segundo plano, la abre sin conexión y muestra los avisos aunque la aplicación esté cerrada.
- Los servicios de Google publican la aplicación con Firebase Hosting, identifican a cada persona con su cuenta, guardan los datos en tiempo real con Cloud Firestore y

entregan los avisos con Firebase Cloud Messaging. El plano de la ciudad lo provee Esri y los recorridos por calles los calcula OSRM, un servicio abierto de ruteo.

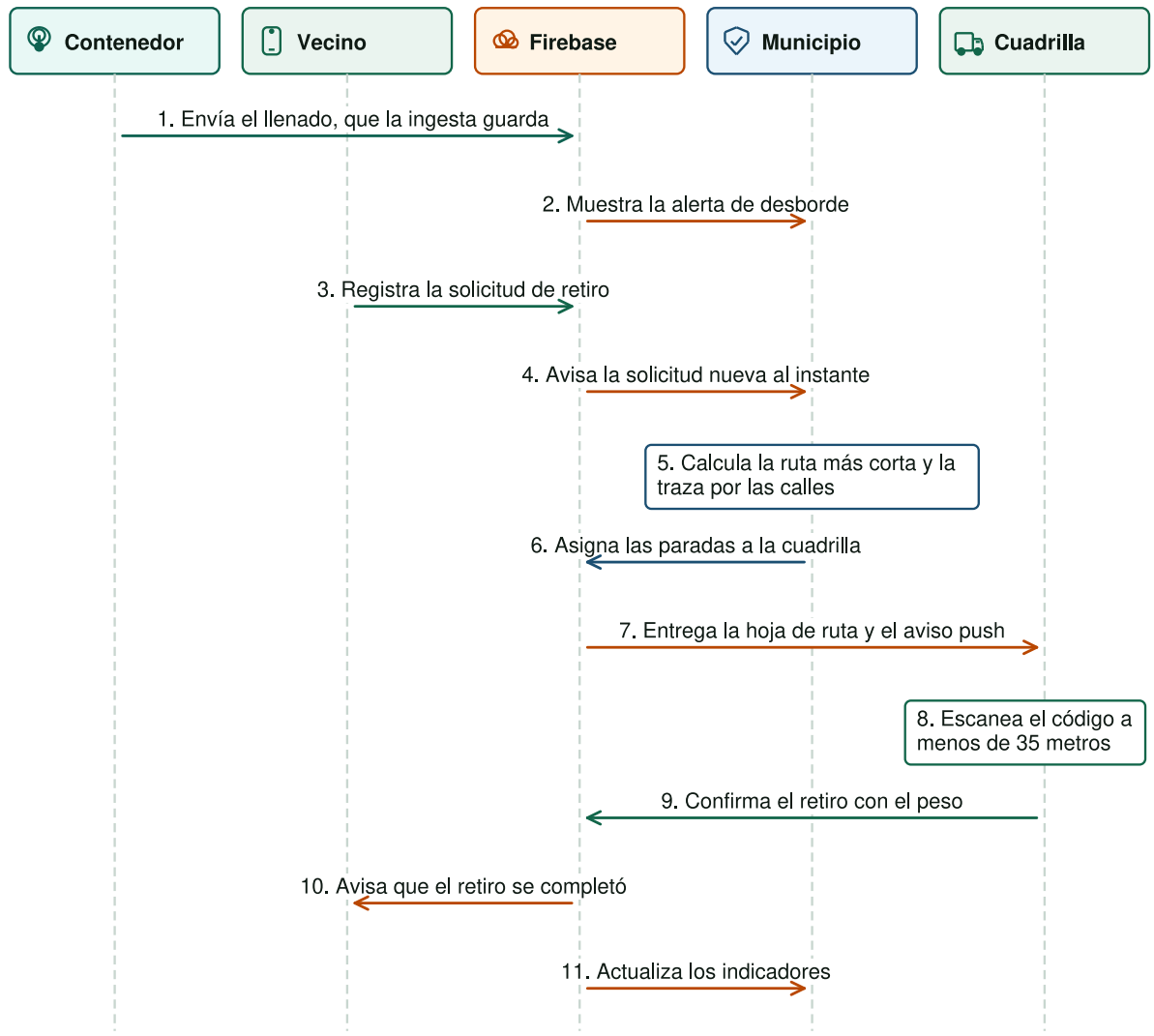
- La infraestructura de la aplicación empaqueta el sitio y su servidor web en una imagen Docker, un paquete que funciona igual en cualquier servidor. Un balanceador reparte el tráfico entre las réplicas, Kubernetes ajusta su cantidad y Prometheus y Grafana vigilan su estado.
- Los sensores de los contenedores envían sus lecturas a Mosquitto, el intermediario de mensajes, y un servicio de ingesta las guarda en Firestore. Ambos corren en el mismo clúster de Kubernetes. Un emisor de avisos envía las notificaciones a los celulares registrados.

Los servicios de Firebase escalan por sí solos porque los administra Google. El clúster de Kubernetes escala lo que aloja el municipio, que es el sitio, el intermediario de mensajes y el servicio que guarda las lecturas. Su configuración se probó en un clúster montado en una computadora.

## 4.2. Flujo de trabajo y secuencia operativa

El circuito de recolección coordina al sensor, al vecino, a la autoridad municipal y a la cuadrilla en la calle. El procedimiento se desarrolla en seis etapas.

- El sensor informa el llenado de su contenedor y el panel municipal muestra una alerta cuando alcanza el 80%.
- El vecino elige la categoría del residuo, indica la cantidad y confirma su ubicación. La solicitud queda registrada como pendiente.
- El panel municipal recibe la solicitud al instante. La autoridad elige las solicitudes que integran la ruta y el sistema calcula el recorrido.
- Al confirmar la ruta, la cuadrilla asignada recibe las paradas en el orden calculado y un aviso en la aplicación.
- En cada parada, la cuadrilla escanea la etiqueta con código QR del contenedor o ingresa el código de la solicitud. El sistema comprueba que el teléfono esté a menos de 35 metros del lugar.
- Al registrar el peso recolectado, el retiro se cierra y el vecino recibe el aviso de que su solicitud se completó.




Secuencia del circuito de alerta, solicitud, asignación y recolección.

## 4.3. Modelo de persistencia y datos en Cloud Firestore

Cloud Firestore guarda los datos porque sincroniza los cambios en tiempo real entre todos los dispositivos y permite definir reglas de seguridad por perfil. La información se organiza en colecciones.

- Solicitudes. Registran quién pidió el retiro, el tipo de residuo, la cantidad, la ubicación, la fecha, el estado, la cuadrilla asignada y el lugar donde se confirmó.
- Registros de residuos. Guardan la categoría y la cantidad que cada vecino anota, sin pedir un retiro.

- Incidentes. Registran cada problema reportado en la calle, su gravedad, su ubicación y su estado.
- Notificaciones. Guardan las campañas que el municipio difunde a la comunidad.
- Lecturas de los sensores. Contienen el llenado, la temperatura y la batería de cada contenedor.
- Dispositivos. Asocian cada celular que aceptó los avisos con su usuario.
- Roles. Indican qué cuentas de Google son autoridad o cuadrilla, y solo las lee su propio titular.

Los vecinos crean y consultan solo sus solicitudes y sus registros, y pueden cancelar una solicitud mientras sigue pendiente. La autoridad lee y modifica todo.

La cuadrilla cambia únicamente los campos de su tarea, como el estado de la parada, el peso y el lugar de confirmación, y puede tomar o cerrar un incidente. Las reglas de seguridad comprueban el tipo y el valor de cada campo antes de aceptarlo.

Los datos de cada ecopunto, como su dirección y su horario, forman parte de la aplicación y cambian con una nueva versión. Su llenado, en cambio, llega en vivo desde los sensores.

## 4.4. Algoritmo de optimización de rutas de recolección

Ordenar las paradas de una cuadrilla equivale al problema del viajante, que consiste en hallar el recorrido más corto que pasa por todos los puntos. Su solución exacta es demasiado lenta para calcularse al instante con muchas paradas, por lo que se combinan dos métodos que dan buenos resultados rápidamente.

- El primero, conocido como vecino más cercano, parte del punto de salida del camión y visita siempre la parada pendiente más cercana a la posición actual.
- El segundo, una mejora por intercambio de tramos llamada 2-opt, revisa ese recorrido e invierte tramos mientras eso acorte la distancia total.

En 1.000 recorridos de prueba con paradas al azar dentro de la ciudad, el segundo método acortó el recorrido un 4,9% con 5 paradas, un 6,2% con 8 y un 7,5% con 12.

El orden se calcula con la distancia en línea recta. Después OSRM traza el recorrido por las calles reales de San Francisco y calcula los kilómetros que hará el camión. La hoja de ruta se recalcula al instante cuando llega una parada nueva o un contenedor cambia de llenado.

## 4.5. Sensores en los contenedores y protocolo de comunicación

Cada contenedor incorpora un microcontrolador de bajo consumo con un sensor ultrasónico, que mide qué tan lleno está, y una sonda de temperatura.

Las lecturas viajan por MQTT, un protocolo liviano en el que cada sensor publica sus datos y los servicios interesados se suscriben a ellos. El sensor no necesita conocer a quién le llega el dato, y la cabecera mínima del protocolo alarga la vida de la batería.

El siguiente cuadro resume la comparación que fundamenta la elección.

| Protocolo | Funcionamiento | Consumo de datos | Decisión |
|---|---|---|---|
| MQTT | Los sensores publican mensajes en canales y los interesados se suscriben, sobre una conexión permanente | Cabecera mínima de 2 bytes | Elegido por su bajo consumo, la entrega confirmada y la reconexión automática |
| CoAP | Consulta y respuesta binaria sobre mensajes sueltos | Cabecera de 4 bytes | Descartado porque no ofrece aviso de desconexión ni un intermediario que reparta los mensajes |
| HTTP | Cada lectura es una consulta que espera respuesta | Más de 500 bytes por mensaje | Descartado por el consumo de batería |
| LoRaWAN | Radio de largo alcance con mensajes muy cortos | Velocidad que baja con la distancia | Propuesto para zonas sin cobertura celular, mediante una antena que reenvía a MQTT |

Comparación de protocolos de comunicación para los sensores.

Dos mecanismos aseguran la llegada de los datos.

- La entrega con confirmación asegura que el intermediario reciba cada lectura y cada alarma.
- El intermediario publica automáticamente un aviso de desconexión cuando pierde contacto con un sensor, por falta de batería o de señal.

Los canales de mensajes se organizan por contenedor y separan las mediciones periódicas, las alarmas urgentes y las órdenes del municipio, como cambiar la frecuencia de medición. Cada mensaje informa el contenedor, la hora, la distancia medida, el porcentaje de llenado, la temperatura y la batería.

Mosquitto solo acepta conexiones cifradas y con usuario, y cada usuario usa únicamente sus canales. Un sensor publica lecturas pero no puede enviar órdenes.

El servicio de ingesta corre en dos réplicas y Mosquitto reparte los mensajes entre ellas. Cuando un contenedor alcanza el 80%, el servicio registra la alarma y la parada aparece como urgente en la hoja de ruta.

El circuito se probó con sensores simulados, en una computadora y en el clúster montado en una computadora. El servicio guardó cada lectura en Firestore, registró las alarmas y recibió el aviso de desconexión al apagar un sensor simulado.

# 5. PROTOTIPO FUNCIONAL

## 5.1. Módulo de autenticación, perfiles y control de acceso

Firebase Authentication identifica a cada persona con su cuenta de Google. La aplicación no maneja contraseñas propias, y cada sesión recibe una credencial firmada que Cloud Firestore verifica en sus reglas de seguridad.

El perfil de cada cuenta figura en una lista guardada en el servidor, que solo el municipio modifica. Las cuentas de la autoridad y de las cuadrillas están en esa lista, y cualquier otra cuenta entra como vecino. Hay tres perfiles.

- El vecino registra residuos, pide retiros, consulta ecopuntos y horarios, y ve su aporte en kilos. No accede a funciones operativas.
- La autoridad municipal supervisa los contenedores en vivo, despacha rutas y atiende incidentes.
- La cuadrilla consulta su hoja de ruta, escanea códigos QR y registra el peso recolectado.

Cada perfil tiene su propio tablero y no existe un selector de perfil en la navegación. Para cambiar de perfil se cierra la sesión y se ingresa con otra cuenta.

El siguiente cuadro resume lo que cada perfil puede hacer. Las reglas de seguridad del servidor lo aplican aunque alguien intente acceder a los datos sin pasar por la aplicación.

| Acción | Vecino | Autoridad | Cuadrilla |
|---|---|---|---|
| Registrar residuos y pedir retiros a su nombre | Sí | Sí | Sí |
| Ver las solicitudes de todos los vecinos | No | Sí | Sí |
| Asignar paradas y despachar rutas | No | Sí | No |
| Confirmar la recolección y cargar el peso | No | Sí | Sí |
| Reportar un incidente | Sí | Sí | Sí |
| Tomar o cerrar un incidente | No | Sí | Sí |

| Acción | Vecino | Autoridad | Cuadrilla |
|---|---|---|---|
| Enviar una campaña a la comunidad | No | Sí | No |
| Registrar el vaciado de un contenedor | No | Sí | Sí |

Permisos de cada perfil sobre los datos del sistema.

## 5.2. Módulo ciudadano

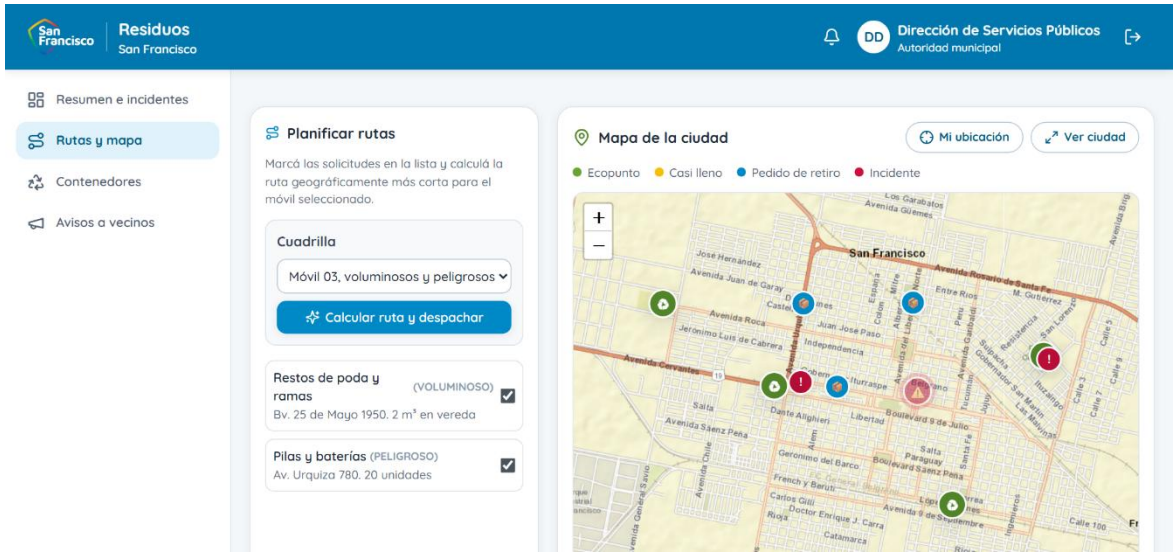
La aplicación del vecino evita formularios largos que desalienten la participación y se adapta al celular.

- El registro de residuos anota lo que el vecino separó en su casa, con su categoría y su cantidad en kilos, sin pedir un retiro. El portal muestra lo registrado en el mes.
- La solicitud de retiro ofrece reciclables, voluminosos y peligrosos, con subcategorías como papel y cartón, restos de poda o pilas y baterías. Pide la cantidad aproximada, la dirección, la ubicación en el mapa y la fecha deseada.
- El mapa muestra los ecopuntos con su dirección, su horario de atención, los materiales que reciben y su llenado. Una lista indica si cada uno está disponible, casi lleno, por encima del 60%, o lleno, desde el 80%.
- Una tabla informa por zona los días y horas de recolección domiciliaria, de reciclables y de retiros especiales, y resalta la zona del vecino.
- El centro de avisos muestra las campañas municipales al instante, y el historial de pedidos muestra el avance de cada uno.

## 5.3. Panel de control para autoridades municipales

El panel concentra las herramientas de supervisión y despacho de la autoridad municipal.

- Los indicadores resumen las solicitudes pendientes, las cuadrillas con retiros asignados, los incidentes abiertos y los ecopuntos activos.
- La vista de contenedores muestra el llenado, la temperatura y la batería de cada uno, con actualización en vivo. También permite registrar una medición a mano si un sensor falla.
- El despachador calcula el recorrido optimizado de las solicitudes marcadas, lo traza por las calles y lo asigna a la cuadrilla elegida, que lo ve al instante en su aplicación.
- La consola de incidentes lista los problemas reportados, los clasifica por gravedad y permite marcarlos como resueltos.
- Las campañas se redactan en el panel y llegan en vivo a vecinos y cuadrillas.




Panel municipal en escritorio, con sus secciones en un menú lateral.

## 5.4. Módulo operativo para personal de cuadrilla

La aplicación de la cuadrilla se diseñó para usarse con una mano y en movimiento, con textos legibles a pleno sol.

- La hoja de ruta presenta las paradas en el orden calculado, el recorrido por calles con sus kilómetros y un resumen del turno. Incluye los contenedores con más de un cuarto de su capacidad ocupada y marca como urgentes los que alcanzan el 80%.
- El escáner lee con la cámara la etiqueta con código QR de cada contenedor o acepta el código escrito a mano. En un retiro a domicilio se usa el código de la solicitud, que el vecino también ve en su portal.
- Al tocar *Confirmar* en una parada se abre una ventana con el código, el peso y las observaciones, sin salir de la hoja de ruta.
- El reporte de problemas se completa con dos toques y llega al instante al panel municipal.

Antes de cerrar la parada, la aplicación comprueba que el código exista y que el teléfono esté a menos de 35 metros. Si está más lejos o no informa su ubicación, pide una confirmación y registra en la base dónde se hizo la recolección.

## 5.5. Capturas de la interfaz en ejecución

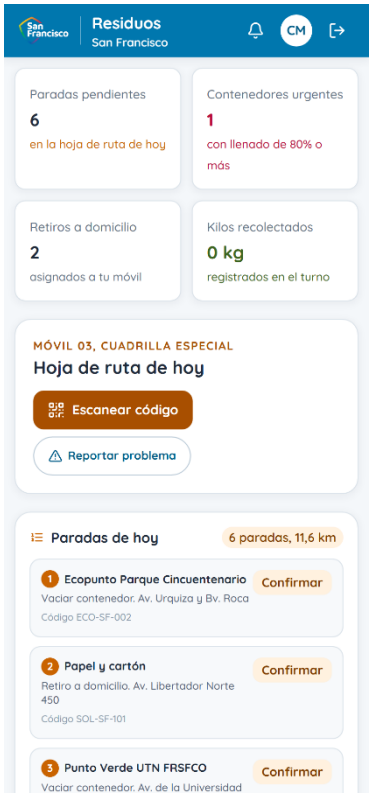
Las capturas muestran el portal del vecino, el detalle de un ecopunto y la hoja de ruta de la cuadrilla en el celular.




Portal del vecino.




Horario y llenado de un ecopunto.




Hoja de ruta de la cuadrilla.

## 5.6. Plataforma de cartografía y geolocalización

El mapa se construye con la biblioteca Leaflet sobre el plano de Esri, sin marcas de agua ni cargos por uso. Google Maps y Mapbox, que cumplen la misma función, exigen una clave y cobran a partir de cierto volumen de uso. Si OSRM no responde, el mapa une las paradas con líneas rectas.

El mapa se centra en San Francisco y distingue por color los ecopuntos, los contenedores casi llenos, los pedidos de retiro y los incidentes. Al tocar un ecopunto se ven su horario, los materiales que recibe y su llenado. Un botón centra la vista en la ubicación del teléfono.

# 6. ESCALADO Y MONITOREO

## 6.1. Contenerización de servicios con Docker

La aplicación se empaqueta en una imagen basada en el servidor web Nginx sobre una versión reducida de Linux, que se ejecuta sin permisos de administrador.

| Atributo | Imagen estándar de Nginx | Imagen del proyecto | Beneficio |
|---|---|---|---|
| Tamaño en disco | 243 MB | 30,2 MB | Reduce un 88% la descarga de cada réplica nueva |
| Usuario de ejecución | Administrador del sistema, con todos los permisos | Usuario sin privilegios | Limita el daño si alguien vulnera el contenedor |
| Contenido | Servidor genérico con configuración por defecto | Solo la aplicación y su configuración de seguridad | Reduce los puntos de ataque |

Comparación entre la imagen estándar y la imagen del proyecto.

El contenedor incluye dos controles automáticos de salud. El primero permite que Kubernetes reinicie la réplica que deja de responder. El segundo hace que una réplica reciba tráfico solo cuando está lista.

## 6.2. Orquestación y balanceo con Kubernetes

Kubernetes ejecuta las réplicas de la aplicación y solo las expone a internet a través de la puerta de entrada del clúster.

- La puerta de entrada funciona como balanceador de carga. Recibe el tráfico de internet, exige conexión cifrada, limita a 50 las peticiones por segundo de cada equipo y reparte el resto entre las réplicas.
- El servicio interno ofrece un punto de acceso estable a las réplicas, aunque se creen o se retiren.
- La actualización progresiva agrega una réplica nueva antes de retirar la anterior, de modo que los vecinos no pierden el servicio durante un cambio de versión.
- La distribución entre servidores procura ubicar las réplicas en máquinas distintas, para que la caída de una no detenga la aplicación.

El siguiente cuadro compara las opciones evaluadas para alojar el servicio.

| Opción | Ventajas | Desventajas | Decisión |
|---|---|---|---|
| Kubernetes con autoescalado | Reparte la carga, reemplaza las réplicas caídas y escala sin intervención | Requiere administrar un clúster | Elegida para la infraestructura propia del municipio |
| Servidores virtuales fijos | Configuración simple | Escalar exige trabajo manual y se paga la capacidad ociosa | Descartada |
| Contenedores administrados por un proveedor, como Google Cloud Run | Escalan solos sin administrar un clúster | Atan el servicio a un proveedor y limitan el control de la red | Alternativa si el municipio no administra servidores |
| Solo Firebase Hosting | Sin servidores que mantener y con distribución mundial | No aloja Mosquitto ni el servicio de ingesta | Usada para publicar el prototipo |

Opciones evaluadas para alojar el servicio.

## 6.3. Autoescalado horizontal de réplicas

El autoescalado ajusta la cantidad de réplicas según el uso del procesador que mide el clúster.

- Cuando el procesador supera el 70% de lo reservado para cada réplica, agrega réplicas.
- Nunca baja de 2 réplicas, aun con poca actividad, ni supera las 10, el techo previsto para emergencias o campañas.
- Ante un aumento brusco, puede duplicar las réplicas cada 15 segundos para no perder velocidad de respuesta.
- Al bajar la demanda, espera 300 segundos antes de retirar réplicas, para evitar encendidos y apagados continuos.

La política se comprobó en ese clúster con seis programas que enviaban peticiones sin pausa. El uso del procesador se mide sobre lo reservado para cada réplica, por lo que puede superar el 100%.

| Tiempo desde el inicio de la carga | Réplicas en funcionamiento | Uso del procesador |
|---|---|---|
| 0 segundos | 2 | 1% |
| 30 segundos | 2 | 73% |
| 45 segundos | 2, y el autoescalado ya pidió 6 | 491% |
| 60 segundos | 6 | 470% |
| 80 segundos | 10 | 433% |
| 100 segundos | 10 | 145% |

Respuesta del autoescalado ante una carga sostenida.

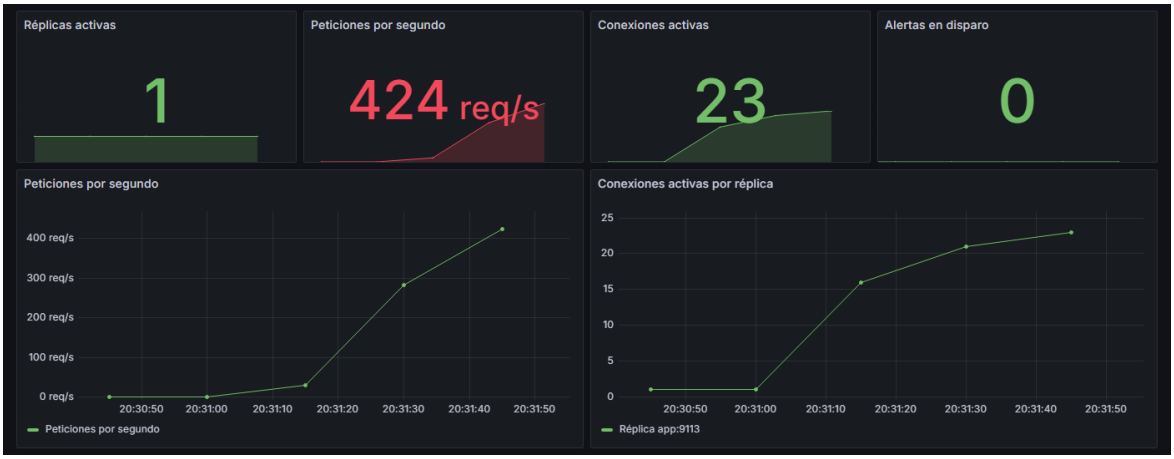
Las réplicas nuevas tardan unos segundos en arrancar. Con 10 el uso sigue en 145%, así que el techo puede ampliarse si la demanda lo exige.

## 6.4. Observabilidad y monitoreo con Prometheus y Grafana

Cada réplica incorpora un exportador, un programa auxiliar que publica las métricas del servidor Nginx. En el clúster, Prometheus las recolecta cada 15 segundos y Grafana las presenta en un tablero.

- El tablero muestra las réplicas activas, las peticiones por segundo y las conexiones abiertas, con actualización cada 5 segundos.
- Una alerta crítica se activa si una réplica deja de informar durante 30 segundos.
- Una advertencia se activa si el tráfico supera las 200 peticiones por segundo durante un minuto o si las conexiones abiertas superan las 400.
- Las alertas se ven en el tablero; para recibirlas por correo se agrega el componente de reenvío de Prometheus.

En la prueba con 60 clientes durante 60 segundos, el servidor atendió 432 peticiones por segundo en promedio, con un pico de 456. Al detener el exportador de una réplica, la alerta crítica se activó a los 45 segundos, que suman los 30 de espera y los ciclos de lectura y evaluación.




Tablero de Grafana durante la prueba de carga.

# 7. ESTRATEGIA DE PRUEBAS Y CALIDAD

Las pruebas automáticas recorren cada perfil, comprueban la sincronización entre dispositivos y verifican las reglas de seguridad contra la base publicada. Se ejecutaron sobre la copia local, el contenedor Docker y la aplicación publicada.

| Nivel | Herramienta | Qué se verificó | Resultado |
|---|---|---|---|
| Flujos por perfil | Navegador automatizado | Recorrido completo del vecino, la autoridad y la cuadrilla, y que cada perfil vea solo sus pantallas | Aprobado |
| Datos en vivo | Tres sesiones simultáneas | Solicitudes, despachos, campañas e incidentes que llegan en vivo a los otros perfiles | Aprobado, entre 0,5 y 2,3 s, con demoras aisladas de hasta 23 s |
| Validación de ubicación | Navegador con ubicación simulada | Aceptación dentro de los 35 metros y confirmación expresa fuera de ellos | Aprobado |
| Seguridad | Consultas directas a la base y emulador | Permisos de cada perfil, validación de cada campo y rechazo sin sesión | Aprobado |
| Avisos push | Chrome y Firebase Cloud Messaging | Aviso recibido con la aplicación cerrada y baja del dispositivo al cerrar sesión | Aprobado |
| Sin conexión | Navegador sin red | Apertura de la aplicación con sus bibliotecas guardadas en el teléfono | Aprobado |

| Nivel | Herramienta | Qué se verificó | Resultado |
|---|---|---|---|
| Rutas | 1.000 recorridos por tamaño | Mejora del segundo método sobre el primero | Entre 4,9% y 7,5% |
| Carga | 60 clientes durante 60 segundos | Respuesta del contenedor Docker | 432 peticiones por segundo, ninguna con error y 95% en menos de 214 milisegundos |
| Autoescalado | Clúster en una computadora y seis programas de carga | Aumento de réplicas bajo demanda | De 2 a 10 réplicas en poco más de un minuto |
| Calidad web | Lighthouse, la auditoría de calidad de Chrome | Accesibilidad, buenas prácticas, posicionamiento en buscadores y rendimiento | Accesibilidad 100, buenas prácticas 100, buscadores 92, rendimiento 100 en escritorio y entre 67 y 87 en celular |
| Sensores | Mosquitto y simulador de contenedores | Lecturas guardadas en Firestore, alarmas, aviso de desconexión y rechazo de conexiones sin usuario | Aprobado |

Pruebas realizadas sobre el prototipo y sus resultados.

La página completa pesa 505 kilobytes en el celular, dentro del límite de 2 megabytes, porque solo incluye los estilos e íconos que usa y carga el lector de códigos QR al escanear.

# 8. PROBLEMAS Y SOLUCIONES

Durante la integración surgieron dificultades que exigieron ajustes en la arquitectura. El cuadro resume cada problema, su causa y la solución aplicada.

| Problema | Causa | Solución |
|---|---|---|
| El mapa se bloqueaba o mostraba marcas de agua | Los servidores públicos de OpenStreetMap, un mapa colaborativo, rechazaban las peticiones y otro proveedor exigía clave y superponía su marca | Cambio al plano de Esri y ajuste de la política de seguridad del sitio |
| Instrucciones de seguridad ausentes en los archivos estáticos | Nginx descarta las instrucciones generales cuando una sección define las propias | Repetición de las instrucciones en la sección de archivos estáticos |

| Problema | Causa | Solución |
|---|---|---|
| Lectura de códigos QR lejos del contenedor | El escáner aceptaba el código sin comprobar dónde estaba el teléfono de la cuadrilla | Validación de la ubicación a menos de 35 metros, con confirmación expresa fuera del radio |
| Riesgo de inyección de código en los listados | Los textos de los vecinos se insertaban en la página sin filtrar | Función que neutraliza los caracteres peligrosos, y códigos que solo admiten letras, números y guiones |
| Permisos demasiado amplios | El vecino podía cambiar cualquier campo de su pedido pendiente y los incidentes se leían sin sesión | Reglas que validan cada campo, limitan los cambios por perfil y exigen sesión |
| Intermediario de mensajes abierto | Mosquitto aceptaba conexiones sin usuario ni cifrado | Conexión cifrada, usuario por servicio y permisos por canal |
| Solicitudes que no llegaban a la nube | Las cuentas guardadas solo en el navegador no creaban una identidad que las reglas aceptaran | Ingreso con Firebase Authentication y cuentas de Google |
| Cuentas de demostración públicas | Las claves de la autoridad y de la cuadrilla estaban en el código y cualquiera podía usarlas | Perfiles por cuenta de Google guardados en el servidor y eliminación de las cuentas de prueba |
| Monitoreo sin métricas | Las réplicas no exponían datos que Prometheus pudiera leer | Exportador de métricas de Nginx en cada réplica y alertas sobre esas métricas |

Problemas detectados, causas y soluciones adoptadas.

# 9. CUMPLIMIENTO DE LA CONSIGNA

El siguiente cuadro relaciona cada requerimiento de la consigna con la funcionalidad que lo cubre.

| Requerimiento | Cómo se cumple |
|---|---|
| Registro de residuos generados por categoría y cantidad | Registro propio del vecino, sin pedir retiro, con el total del mes |
| Solicitud de recolección de reciclables, voluminosos y peligrosos | Solicitud con categoría, cantidad, dirección, ubicación en el mapa y fecha deseada |
| Puntos de reciclaje, horarios y campañas | Mapa con dirección, horario, materiales y llenado de cada ecopunto, horarios por zona y avisos de campañas |
| Puntos a recolectar y su estado en tiempo real | Panel municipal con contenedores y solicitudes que se actualizan al instante |
| Planificación y asignación de rutas | Recorrido optimizado con dos métodos combinados, trazado por las calles y asignado a una cuadrilla |

| Requerimiento | Cómo se cumple |
|---|---|
| Monitoreo de los puntos y gestión de incidentes | Llenado, temperatura y batería de cada contenedor, alerta al 80% y consola de incidentes |
| Rutas asignadas y confirmación de tareas | Hoja de ruta ordenada y confirmación de cada parada con peso y observaciones |
| Escaneo de códigos QR o ingreso de identificadores | Lectura con la cámara o código escrito a mano, con validación a 35 metros |
| Reportes sobre contenedores e incidentes | Reporte de problemas en dos toques que llega al panel municipal |
| Mapeo y geolocalización | Mapa con Leaflet y Esri, ubicación del teléfono y recorridos por calles con OSRM |
| Notificaciones push para vecinos y cuadrillas | Avisos en vivo y notificaciones que llegan con la aplicación cerrada |
| Contenedores Docker y Kubernetes | Imagen de 30,2 MB, balanceador de entrada con conexión cifrada y réplicas |
| Monitoreo y autoescalado | Prometheus, Grafana y alertas, con autoescalado de 2 a 10 réplicas comprobado bajo carga |

Cumplimiento de los requerimientos de la consigna.

El repositorio https://github.com/facupepi/arquitecturas-moviles-residuos-urbanos reúne todo lo necesario para reproducir la solución. Contiene la aplicación, la imagen Docker y la configuración de Kubernetes, las reglas de seguridad, el monitoreo, el circuito de sensores y los resultados de cada medición.

La aplicación publicada se encuentra en https://sf-residuos-urbanos-utn.web.app. El circuito completo se recorre con tres ventanas del navegador, una por perfil, y un celular para los avisos.

- Vecino. Con cualquier cuenta de Google, registra lo que generó, pide un retiro de residuos voluminosos y revisa el mapa, los horarios y los contenedores.
- Autoridad. Con una cuenta habilitada por los autores a pedido, ve llegar la solicitud, despacha la ruta y envía una campaña.
- Cuadrilla. Con otra cuenta habilitada, recibe la parada, ingresa el código y confirma el peso.
- Avisos. Con el emisor de avisos en ejecución, las campañas, los cambios de cada pedido y las paradas nuevas llegan al celular aunque la aplicación esté cerrada.

# 10. INNOVACIÓN Y PARTICIPACIÓN

Cuatro mejoras potencian la eficiencia y la participación. La primera ya empezó.

- Un sistema de incentivos canjea los kilos reciclados por descuentos en tasas municipales. El prototipo ya suma los kilos de cada vecino.
- Una predicción del llenado, con el historial de cada sensor, agrega el contenedor a la ruta antes de que desborde.
- Un mapa público muestra los basurales denunciados con foto, para que la comunidad siga su limpieza.
- Una clasificación mensual premia a los barrios que más reciclan.

# 11. CONCLUSIONES

El prototipo muestra que la recolección puede modernizarse con servicios móviles y de nube.

- Firebase atiende los datos y los avisos sin servidores propios, y Kubernetes escala lo que aloja el municipio.
- Las rutas combinadas acortan el recorrido entre un 4,9% y un 7,5%, y la validación en la calle deja registro del trabajo.
- El autoescalado pasó de 2 a 10 réplicas bajo carga, y el monitoreo detectó una réplica que dejó de informar.
- El registro de residuos, los avisos y el seguimiento de cada pedido acercan a los vecinos al cuidado ambiental.

# 12. BIBLIOGRAFÍA

- Bono, J. P. (2026). *Apuntes de Cátedra: Modelos de Arquitecturas basados en Cloud Computing y Escalado de Aplicaciones*. UTN Facultad Regional San Francisco.
- Google. (2026). *Firebase documentation: Cloud Firestore, Cloud Messaging y Hosting*. Firebase.
- Kubernetes Authors. (2026). *Horizontal Pod Autoscaling*. Kubernetes Documentation.
- Leaflet. (2026). *Leaflet API reference*. Leaflet.
- OASIS. (2019). *MQTT Version 5.0*. OASIS Standard.
- Project OSRM. (2026). *OSRM API documentation*. Project OSRM.
- Prometheus Authors. (2026). *Prometheus documentation*. Prometheus.